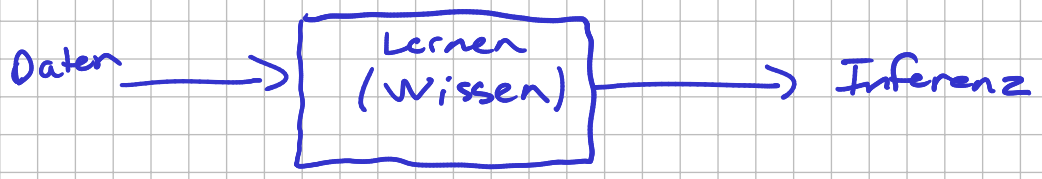
AKIB10 20260417

Wissensrepräsentation



- Ontologien
- Knowledge Graphs
- Symbolische KI

⇒ Was ist Wissen?

Um zu verstehen was Wissen in einem KI-Kontext ist, müssen wir einige verwandten Konzepten unterschieden:

- Daten - rohe Fakten oder Zahlen
- Informationen - Daten mit Kontext
- Wissen - zusammenhängenden Fakten und Schlussfolgerungen

Daten

⇒ Es kann um Sensormesswerte, numerische werte, Pixel-RGB werte, u.sw.

Bsp: "Temperatur", 20, "Lüfter", "Zeit", 08:30

⇒ Rohe Fakten oder Zahlen!

Information

⇒ Sobald den Daten ein gewisser Kontext gegeben wird werden sie zu Informationen

Beispiele:

- "Die um 08:30 Uhr gemessene Temperatur betr. 20.0 Grad"
- "Der Lüfter wurde um 08:30 Uhr eingeschaltet."

Wissen

⇒ Wissen wird aus Daten abgeleitet, indem Fakten durch Schlusserfolgerungen miteinander verbunden werden.

Bsp: "Wenn die Temperature 20 Grad erreicht, schalten sie den Lüfter an."

Datenrepräsentation (DR)

⇒ Die DR hat das Ziel, rohe Fakten zu speichern und zu bearbeiten. DR hat eine niedrige Ebene (Low-level), da es mit Bits, Strings, und Tabellen arbeitet.

⇒ Daten können in vielen Formate gespeichert werden, darunter:

- CSV
- JSON
- Binärdaten.

⇒ DR wird in der Datenbankentwicklung, bei Sensoren, und APIs eingesetzt.

Wissensrepräsentation (WR)

⇒ WR hat das Ziel, auf der Grundlage von Daten Schlusserfolgerungen (Reasoning) und Verständnis zu ermöglichen.

⇒ WR ist hochrängig, da sie sich mit Konzepten, Regeln und Logik befasst.

⇒ Wissen kann in viele Formate dargestellt werden:

- Graphen
- logische Regeln
- Symbole und Neuronale Gewichte

⇒ WR ist ein grundlegender Bestandteil der KI und wird in Expertsysteme, Chatbots, KI-Agenten und in LLMs.

DR vs. WR

Analogie:

DR – eine Bibliothek mit Büchern (rohe Fakten/Infos)

WR – eine Bibliothekar der weiß was in der Bücher steht, und kann Fragen beantworten.

Beispiele

DR in CSV-Format:

Wert,	Sensor/Aktor,	Zeit
"20",	"Temperatur",	"08:30"
"1",	"Lüfter",	"08:30"

WR in Logik:

$T \rightarrow L$

"Wenn der Temperatur 20° erreicht, dan schalten den Lüfter an."

Ontologien

Definition: Eine Formale Darstellung von Wissen als eine Menge von Konzepten und Beziehungen zwischen ihnen.

Zweck: Ermöglichung eines gemeinsam Verständnisses, semantischer Schlussfolgerungen und interoperabilität.

⇒ In der Informatik und KI definiert eine Ontologie:

- Welche Arten von Dinge es gibt
- Wie sie sich zueinander verhalten

Bestandteile von Ontologien

Ontologien bestehen aus:

Klassen – sind Kategorien oder Arten von Dingen

Instanzen – (Individuals) sind tatsächliche Dinge, die zu Klassen gehören – Beispiele aus der realen Welt.

Eigenschaften – (Relationen) definieren, wie sich Instanzen zueinander verhalten, oder welche Eigenschaften sie haben.

Beispiel – Klassen: Klassen sind astrakte konzepten von Dinge

- Person
- Stadt
- Universität
- Sensor

Beispiel – Instanzen: Instanzen sind ihrer Datenpunkte im Ontologie

- Hendrik (eine Instanz von Person)
- Berlin (eine Instanz von Stadt)
- HHN (Instanz von Uni)
- Temperatursensor (Instanz von sensor)

Eigenschaften – Es gibt 2 Haupttypen von Eigenschaften:

⇒ Objekt-Eigenschaften – Beziehen eine Instanz auf eine andere

Bsp: studiertAn(Hendrik, HHN)

⇒ Daten-Eigenschaften – Verknüpfen eine Instanz mit einem Wert

Bsp: hatWert(Temperatursensor, 20)

Web Ontology Language (OWL)

OWL ist eine W3C entwickelte Standardspache zur Definition und Austausch von Ontologien im Web. OWL wird verwendet, um Argumentation, Interoperabilität, und maschinelles Verständnis komplexer Daten zu ermöglichen.

Resource Description Framework (RDF)

⇒ OWL baut auf RDF auf und verwendet RDF-triples als zugrunde liegendes Datenmodell.

⇒ OWL ontologies are ultimately expressed as RDF-triples:

<Subjekt, Prädikat, Objekt>

Bsp:

:Student rdfs:subClassOf :Person

→ Dies ist ein Beispiel für ein Relation zwischen zwei Klassen, beschrieben als RDF-Triple.

→ Dies ist ein Form der Wissensrepräsentation

Domäne (Domain)

⇒ Eine Domäne liegt fest, welche Art von Dinge (Klasse) das Subjekt sein muss.

Bsp: :Hendrik rdfs:studiertAn :HHN

→ Die Klasse oder Domäne von der Subjekt Hendrik ist

$\boxed{\text{Person}}$

Wertebereich (Range)

⇒ Eine Wertebereich liegt fest, welche Art von Dinge (Klasse) das Objekt sein muss.

Bsp: :Hendrik rdfs:studiertAn :HHN

→ Die Klasse oder Wertebereich von der Objekt HHN is

$\boxed{\text{Universität}}$

OWL Ontologie Beispiel

Klassen

- Person
- Student (subklass von Person)
- Lehrer (subklass von Person)
- Modul

Eigenschaften

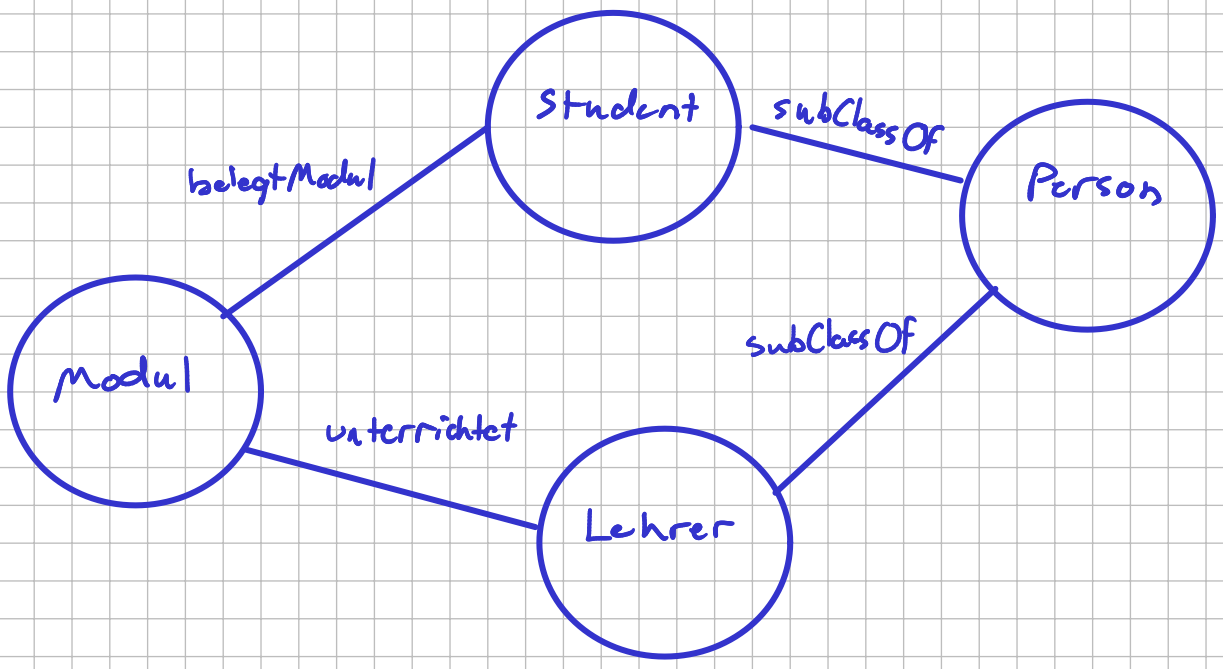
- Belegtmodul
- Unterrichtet

⇒ Wir können diese Ontologie als ein Menge von RDF-triples beschreiben. Hier sind die Klassen:

- :Person rdf:type owl:Class
- :Student rdfs:subClassOf :Person
- :Lehrer rdfs:subClassOf :Person
- :Modul rdf:type owl:Class

... und die Eigenschaften:

- :belegtModul rdf:type owl:ObjectProperty
  rdfs:domain :Student
  rdfs:range :Modul
- :unterrichtet rdf:type owl:ObjectProperty
  rdfs:domain :Lehrer
  rdfs:range :Modul



Knowledge Graphs

Definition: Eine Knowledge Graph (KG) ist eine Ontologie, zusammen mit ihren Instanzen und Daten als Graph dargestellt wird.

KG = Ontologie (Bauplan) + Instanzen (Daten) + GraphenStruktur

⇒ KGs sind auf der Konzept der RDF-triples basiert.

<Subjekt, Prädikat, Objekt>

KG Analogie → Datenbank

⇒ In Datenbanken (SQL z.B) definieren das Schema die Struktur wie ein Bauplan. Es liegt fest welche Tabellen es gibt, welche Spalten sie haben, und welche Datentypen zulässig sind.

⇒ Eine Datenzeile (Record) sind die eigentlichen Daten - eine bestimmte Satz von Werten, die das Schema ausfüllen.

Schicht	Datenbank
Ontologie	Schema
Instanzen	Datenzeile
Knowledge Graph	Datenzeile + Schema

Graphen

KGs sind dargestellt durch gerichteten, beschrifteten Graphen. Ein Graph ist ein Struktur, die ein Netzwerk beschreibt. Er ist mathematisch als Tuple definiert, mit eine Menge von Nodes (oder Knoten) und Edges (Verbindungen).

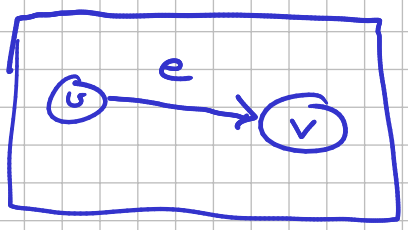
Graph: $G = (V, E)$

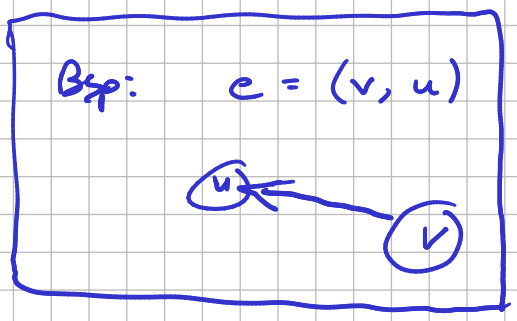
Nodes: $V = \{v_1, v_2, \dots\}$

Edges: $E = \{e_1, e_2, \dots\}$

$\Rightarrow$ Für nodes u und v in einem gerichteten Graphen definieren wir eine Edge von u nach v als:

$$\boxed{e = (u, v) \quad \text{für } u, v \in V}$$





RDF-Triples → KG Implementation

Ein <u>Triple-Store</u> ist eine Datenbank, die für die Speicherung von RDF-Triples konzipiert ist.

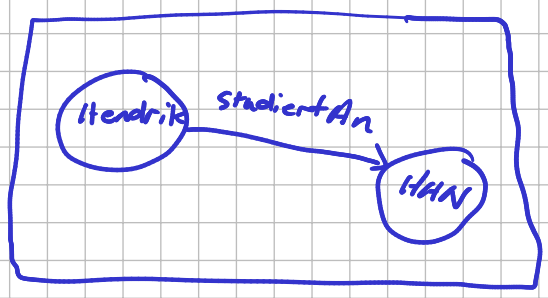
⇒ Eine RDF-Triple wird als gerichtete Edge (Prädikat) zwischen einem Node-Paar (Subjekt und Objekt) ausgedrückt.

⇒ Eine KG Node steht für eine Klasse oder Instanz, während ein Edge eine Eigenschaft darstellt.

Bsp:

:Hendrik rdfs:studiertAn :HHN

$e_{studiertAn}$ = (Hendrik, HHN)



KG Beispiel – Autonomes Fahren

<u>Triple-Store Klassen</u>

FZG → :Fahrzeug rdf:type rdf:Class

FZGT → :Fahrzeugteil rdf:type rdf:Class

SYSZ → :Systemzustand rdf:type rdf:Class

SYSK → :Systemkomponent rdf:type rdf:Cla

SNSR → :Sensor rdfs:subClassOf :Systemkomponent

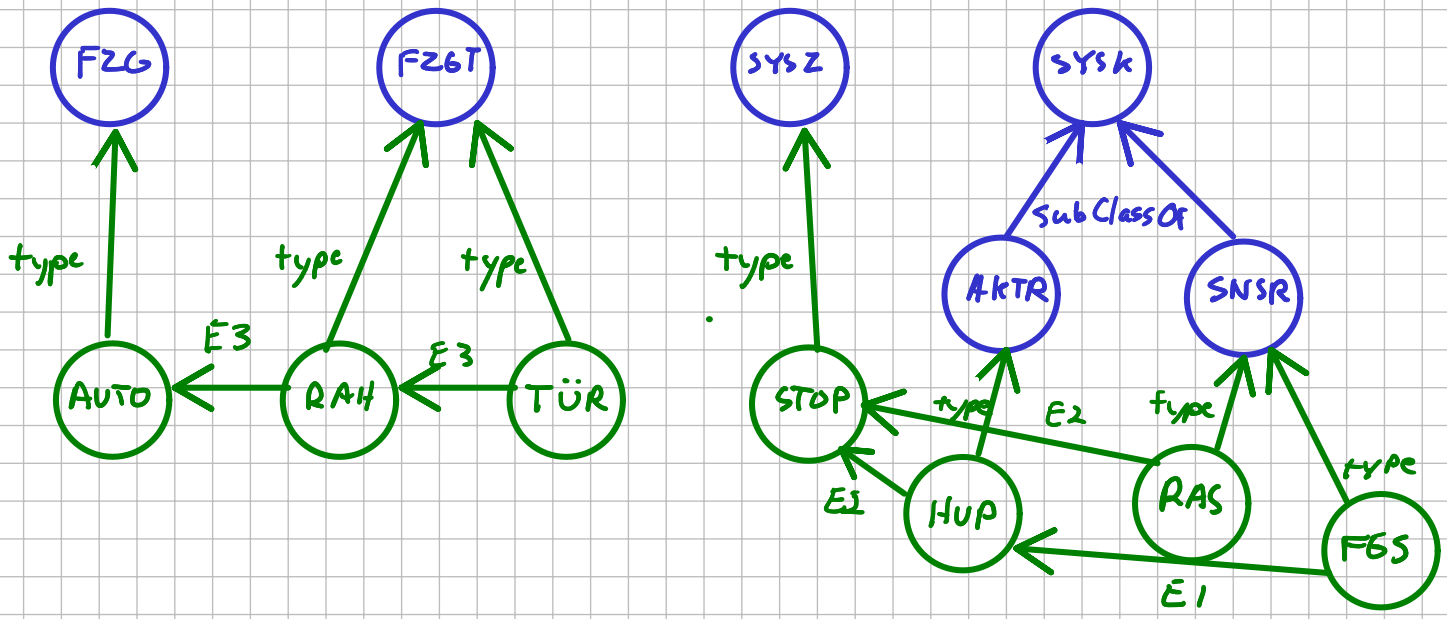
AKTR → :Aktor rdfs:subClassOf :Systemkomponent

<u>Triple-Store Eigenschaften</u>

E1 → löstAus rdf:type rdf:Property;
rdfs:domain :Sensor
rdfs:range :Aktor

E2 → veranlasst rdf:type rdf:Property;
rdfs:domain :Aktor
rdfs:range :Systemzustand

E3 → istTeilVon rdf:type rdf:Property;
rdfs:domain :Fahrzeugteil
rdfs:range :[a owl:Class owl:unionOf (:Fzt :Fzg)].

Triple-Store Instanzen

AUTO → :AutonomesAuto rdf:type :Fahrzeug
RAH → :Rahmen rdf:type :Fahrzeugteil
TÜR → :Tür rdf:type :Fahrzeugteil
FGS → :FussgaengerSensor rdf:type :Sensor
RAS → :RoterAmpelSensor rdf:type :Sensor
HUP → :Hupe rdf:type :Aktor
STOP → :Stop rdf:type :Systemzustand